PLANO TÉCNICO · MY SUMMER CAR

MSC Duo

Multiplayer P2P com progresso individual

Dois jogadores no mesmo mundo, cada um com a própria Satsuma, casa, garagem, dinheiro, necessidades e progresso de rally, com interação física total entre os dois.

Nome do projeto provisório · Versão 1.0 · 14 de outubro de 2026

Documento de planejamento. A seção legal não é aconselhamento jurídico.

CONTEÚDO

Sumário

01	**Resumo executivo**	a ideia, a recomendação e o tamanho do projeto
02	**Requisitos**	tudo que foi combinado, em forma de lista verificável
03	**MSCLoader a fundo**	como ele entra no jogo, ciclo de vida, APIs e regras
04	**Como o My Summer Car funciona por dentro**	PlayMaker, variáveis globais, save ES2
05	**O que já existe**	MSCMP, BeerMP/WreckMP, MSC Online
06	**Arquitetura proposta**	camadas, identidade de objetos, dono e autoridade
07	**Duplicação do mundo do jogador**	casa clonada, Satsuma, garagem, variáveis, save
08	**Rede, lag e física**	interpolação, extrapolação, colisões, autoridade
09	**Mundo compartilhado x individual**	o que é do mundo e o que é de cada jogador
10	**Rally e eventos**	corrida juntos, resultado separado
11	**Roadmap de implementação**	fases, entregas e critérios de pronto
12	**Riscos técnicos**	o que pode dar errado e o plano B
13	**Análise legal**	se vocês podem ser barrados
14	**Ferramentas, setup e próximos passos**	

SEÇÃO 01

Resumo executivo

A ideia é um mod para My Summer Car que coloca **dois jogadores no mesmo mundo, conectados em P2P**, mas com **progresso totalmente individual**: duas Satsumas, duas casas, duas garagens, dinheiro, fome, sede, fadiga, contas, inspeção, encomendas e rally separados. Ao mesmo tempo, os dois podem **interagir fisicamente com tudo**, inclusive com a Satsuma e os itens do outro, e os carros colidem de verdade.

A segunda casa não é modelada do zero. Ela é **clonada em tempo de execução** a partir da casa original, dentro da cópia do jogo de cada jogador, e posicionada em frente a ela.

Decisões recomendadas

Tema	Recomendação	Motivo
Plataforma do mod	MSCLoader (C#, .NET 3.5, Unity 5)	Padrão da comunidade; hooks de PlayMaker, save e assets prontos
Topologia de rede	P2P via Steam Networking (com relay da Steam)	Sem servidor próprio, sem problema de NAT, IP protegido
Modelo de autoridade	Dono fixo + autoridade temporária ao dirigir/segurar	Ninguém sente atraso nas próprias ações
Clone da casa	Instantiate da hierarquia original em runtime	Não redistribui assets do jogo; reaproveita tudo
Variáveis do jogador	Interceptar globais do PlayMaker e trocar por contexto	É o coração do 'progresso individual'
Save	Save do mod separado por jogador (SaveLoad do MSCLoader)	O save do jogo continua do dono da sessão
Base de código de rede	Estudar MSCMP (GPL v3); não forkar WreckMP sem licença	Licença clara só no MSCMP

Tamanho do projeto
Este é um projeto de **vários meses** para uma pessoa. A parte mais cara não é a rede, é transformar o estado global do jogo (PlayMaker) em estado por jogador e sincronizar as centenas de peças da Satsuma. O roadmap da seção 11 entrega algo jogável cedo e vai acrescentando camadas, para que cada fase já valide a seguinte.

Sobre o risco legal (seção 13): modding de MSC é praticado abertamente há quase uma década, já houve vários mods multiplayer e a página da Steam não traz EULA própria do jogo. O risco real está em **redistribuir arquivos ou assets do jogo**, **facilitar pirataria**, **cobrar pelo mod** ou **usar código de outro mod sem licença**. O plano evita os quatro.

SEÇÃO 02

Requisitos

Tudo que foi combinado na conversa, em forma verificável. Cada item tem um código para ser citado no roadmap.

Cód.	Requisito	Prioridade
R1	Multiplayer P2P para 2 jogadores (sem servidor dedicado)	Essencial
R2	Progresso 100% individual: dinheiro, fome, sede, fadiga, sujeira, stress, urina, contas, encomendas, inspeção, rally	Essencial
R3	Uma Satsuma por jogador, montável e salva separadamente	Essencial
R4	Uma casa por jogador; a segunda é um clone da original, em frente a ela	Essencial
R5	Uma garagem / área de trabalho por jogador	Essencial
R6	Interação física com itens do outro jogador (pegar, mover, usar)	Essencial
R7	Poder dirigir e mexer na Satsuma do outro (instalar/remover peças)	Essencial
R8	Colisão real entre jogadores e carros	Essencial
R9	Corridas (racha e rally) com resultado justo para cada um	Alta
R10	Modificar o mapa (terreno, acessos à casa 2, objetos novos)	Alta
R11	Lag baixo: ninguém sente atraso nas próprias ações	Alta
R12	Cada jogador mantém o próprio save entre sessões	Essencial
R13	Código autoral (o mod é de vocês), sem copiar código de mods com licença fechada	Essencial

Fora do escopo inicial (pode entrar depois): mais de 2 jogadores, servidor dedicado, mundo persistente com um dos jogadores offline, compatibilidade com outros mods grandes, suporte ao My Winter Car.

SEÇÃO 03

MSCLoader a fundo

Análise feita a partir do código-fonte oficial (github.com/piotrulos/MSCModLoader, licença GPL v3).

3.1 Como ele entra no jogo

O MSCLoader **não altera o Assembly-CSharp.dll do jogo**. Ele usa duas peças:

- **Unity Doorstop**: um `winhttp.dll` colocado ao lado do executável faz o Unity carregar o `MSCLoader.Preloader.dll` antes do jogo. A configuração fica em `doorstop_config.ini`. O preloader suporta as versões 3 e 4 do Doorstop.
- **Harmony (1.x)**: o preloader aplica patches em memória. O ponto de entrada é um *prefix* em `PlayMakerArrayListProxy.Awake`, que chama `ModLoader.Init`. Também há patches em `PlayMakerFSM.Awake` e em `MousePickEvent` (base do raycast unificado).
- Depois disso, o ModLoader carrega cada DLL da pasta Mods, instancia as classes que herdam de `Mod` e chama `ModSetup()`.

> Consequência prática: como tudo acontece em memória, o mod de vocês pode usar **Harmony** para fazer patch em qualquer método do jogo ou do PlayMaker, sem tocar em arquivo nenhum do jogo. É isso que viabiliza interceptar variáveis globais e trocar o 'dono' de um valor (seção 7).

3.2 Estrutura de um mod

Todo mod é uma classe que herda de `MSCLoader.Mod`, com `ID`, `Name`, `Version` e `Author`. Dentro de `ModSetup()` só é permitido registrar funções com `SetupFunction`:

```
public class MscDuo : Mod
{
    public override string ID => "MscDuo";
    public override string Version => "0.1.0";
    public override string Author => "Kauan";

    public override void ModSetup()
    {
        SetupFunction(Setup.OnMenuLoad, Menu);
        SetupFunction(Setup.OnLoad, Load);
        SetupFunction(Setup.PostLoad, AfterAllMods);
        SetupFunction(Setup.OnSave, Save);
        SetupFunction(Setup.Update, Tick);
        SetupFunction(Setup.FixedUpdate, PhysicsTick);
    }
}
```

3.3 Ciclo de vida (enum Setup)

Fase	Quando roda	Uso no MSC Duo
OnMenuLoad	Uma vez, no menu principal	Lobby: criar/entrar na sessão Steam, escolher qual save
OnNewGame	Ao iniciar jogo novo	Gerar o estado inicial do jogador 2

Fase	Quando roda	Uso no MSC Duo
PreLoad	Logo após a cena GAME carregar	Instalar patches Harmony antes das FSMs rodarem
OnLoad	Cena GAME totalmente carregada	Clonar casa/Satsuma, registrar objetos na rede
PostLoad	Depois de todos os mods no OnLoad	Ligar a rede e sincronizar o estado inicial
Update / FixedUpdate	Todo frame / todo passo de física	Envio e recepção de pacotes; física remota
OnGUI	Igual ao OnGUI do Unity	Debug: ping, dono de cada objeto
OnSave	Depois que o jogo salva	Salvar o progresso do jogador 2 e das cópias
ModSettings / OnModEnabled / Disabled	Menu de configurações do mod	Opções: nome, regras do mundo compartilhado

3.4 APIs que o projeto vai usar

API	O que faz	Onde entra
GetPlayMaker(nome) / GetPlayMakerState	Acha uma FSM ou estado num GameObject	Ler e alterar lógica do jogo
FsmInject(fsm, estado, Action)	Injeta uma chamada C# dentro de um estado de FSM	Saber quando algo acontece (pagou, comeu, parafusou)
AddEvent / AddGlobalTransition / AddVariable	Modifica FSMs em runtime	Redirecionar FSMs clonadas para o jogador 2
InitializeFSM	Força a inicialização de uma FSM	FSMs clonadas que ainda não rodaram
MakePickable	Torna um objeto pegável pelo sistema do jogo	Itens novos ou clonados
SaveLoad.WriteValue / ReadValue / SerializeClass	Save do mod (por trás usa o ES2, o mesmo do jogo)	Progresso do jogador 2
SaveLoad.SaveGameObject / LoadGameObject	Salva posição/estado de um GameObject	Itens soltos da casa 2
LoadAssets (AssetBundle)	Carrega bundles feitos no Unity	Objetos novos do mapa (R10)
ModConsole / Keybind / ModSettings	Console, atalhos, configurações	Debug e UX

3.5 Regras e cuidados

- **Divulgação de código gerado por IA**: o próprio `Mod.cs` pede que código gerado por IA seja declarado na descrição do mod e no campo `AssemblyTrademark` do AssemblyInfo. Como a regra de vocês é que o código seja autoral, usem IA para análise e revisão e escrevam o código; se algo vier de IA, declarem.
- **Licença GPL v3**: o MSCLoader é GPL v3. Publicar o mod também como GPL v3 (código aberto) é o caminho sem atrito e combina com um projeto de portfólio.

- **Versão do Unity**: o jogo usa Unity 5.0 (o MSCMP gerava bundles com `Unity 5.0.0f4`). Asset bundles precisam ser gerados exatamente nessa versão.
- **.NET 3.5**: o Mono do Unity 5 só aceita bibliotecas compatíveis com .NET Framework 3.5. Isso limita a escolha de bibliotecas de rede e serialização (seção 14).
- **Erros no Update**: o loader conta erros no Update de cada mod e pode desativá-lo. O código de rede deve tratar exceções internamente.

SEÇÃO 04

Como o My Summer Car funciona por dentro

O que precisa ser entendido antes de duplicar qualquer coisa.

4.1 Lógica em PlayMaker, não em C#

O código C# do jogo é pequeno. A lógica de praticamente tudo (portas, chuveiro, comida, telefone, contas, peças da Satsuma, parafusos, NPCs, rally) fica em **PlayMaker FSMs**: componentes `PlayMakerFSM` com estados, transições e ações. Para descobrir como algo funciona, a ferramenta certa é inspecionar as FSMs em runtime (mods de inspeção/dev mode do MSCLoader), não decompilar o Assembly-CSharp.

4.2 Os três obstáculos para o progresso individual

Obstáculo	Como aparece	Impacto
Variáveis globais	FsmVariables.GlobalVariables guarda dinheiro, fome, sede, fadiga, stress, sujeira e outros (ex.: PlayerMoney)	Existe um único jogador no modelo do jogo
Busca por nome	FSMs e scripts usam GameObject.Find com nomes fixos, como o nome do carro com peso e id	Uma cópia continua mexendo no original
Save monolítico	O jogo salva tudo num arquivo ES2 (Easy Save 2) por tags	Não existe noção de 'save do jogador 2'

4.3 O que o Instantiate preserva

Quando uma hierarquia é clonada com `Object.Instantiate`, o Unity **remapeia as referências internas** (uma FSM da porta clonada aponta para a porta clonada). O que **não** é remapeado: buscas por nome feitas em runtime, variáveis globais, e referências a objetos fora da hierarquia clonada. Isso define exatamente o trabalho de adaptação de cada sistema clonado.

4.4 Static batching

Se a geometria da casa estiver marcada como estática e combinada (static batching), a cópia pode aparecer invisível ou deformada, porque o MeshFilter aponta para uma malha combinada. Isso é a **primeira coisa a validar** (Fase 1). Plano B: reconstruir a malha da cópia a partir dos submeshes, ou carregar a malha a partir dos arquivos do jogo em runtime, sem redistribuí-la.

SEÇÃO 05

O que já existe

Mods multiplayer anteriores e o que dá para aproveitar de cada um.

Projeto	Situação	Licença	Como usar
MSCMP (RootKiller/MSCMP-OLD)	Cancelado; não roda nas versões atuais	GPL v3 (código publicado)	Estudar e reaproveitar ideias e trechos, mantendo GPL v3 e créditos
BeerMP	Descontinuado (relatos de desentendimento de licença)	Não confirmada	Não usar código
WreckMP	Ativo, gratuito, compatível com MSCLoader	Não encontrada publicamente	Usar como referência de jogo; código só com permissão escrita
MSC Online (MSCO)	Pago/pré-venda; reputação mista	Fechada	Não usar

O MSCMP original tinha projetos separados para cliente, injetor e um gerador de mensagens de rede, e empacotava dados em asset bundles do Unity 5.0.0f4. A arquitetura dele é uma boa referência para o formato de mensagens e para a sincronização das peças da Satsuma.

Ponto de diferenciação
Nenhum mod conhecido faz o que vocês querem: **mundo compartilhado com as coisas do jogador duplicadas e progresso separado**. Essa camada é inédita, e é o que vai diferenciar o projeto como portfólio.

SEÇÃO 06

Arquitetura proposta

6.1 Camadas

Camada	Responsabilidade
Transport	Sessão Steam (lobby, convite), envio confiável e não confiável, canais
Protocol	Mensagens tipadas e serialização binária compacta; versionamento do protocolo
Registry	Dá um NetId estável a cada objeto relevante (jogador, carro, peça, item) nos dois PCs
Ownership	Dono permanente + autoridade atual de cada NetId; regras de transferência
Replication	Snapshots de transform/rigidbody, interpolação e extrapolação
Game Bridge	Hooks em FSMs (FsmInject) e patches Harmony que traduzem eventos do jogo em mensagens
PlayerContext	Troca as variáveis globais do jogador ativo; guarda o estado de cada jogador
Duplication	Clona casa/Satsuma/garagem e adapta as FSMs das cópias
Persistence	Save do jogador 2 e das cópias via SaveLoad do MSCLoader

6.2 Identidade dos objetos (NetId)

Os dois PCs precisam concordar sobre 'qual objeto é qual'. Objetos do mapa recebem um id determinístico gerado pelo **caminho na hierarquia** (com índice entre irmãos de mesmo nome). Objetos que nascem durante o jogo (compras, cópias) recebem id do PC que os criou, no formato `dono:contador`, e o outro PC é avisado por mensagem de spawn.

6.3 Dono e autoridade

- **Dono** (permanente): quem salva aquele objeto no próprio save. A Satsuma 1 é do jogador 1 mesmo quando o 2 está dirigindo.
- **Autoridade** (temporária): qual PC simula a física e a lógica daquele objeto agora. Por padrão é o dono.
- **Transferência**: ao entrar no banco do motorista, segurar um item ou parafusar uma peça, o PC pede a autoridade. O detentor atual concede (ou nega se estiver usando). Ao soltar ou sair, a autoridade volta ao dono após um curto atraso.
- **Colisão**: no impacto entre dois corpos de autoridades diferentes, o mais rápido assume a autoridade dos dois por cerca de 0,5 s, para a batida ser calculada num só PC.

6.4 Mensagens principais

Mensagem	Canal	Frequência
PlayerState (posição, rotação, animação, veículo)	Não confiável	20 a 30 Hz
BodySnapshot (NetId, posição, rotação, velocidade linear e angular)	Não confiável	20 a 30 Hz para objetos em movimento

Mensagem	Canal	Frequência
VehicleInput (acelerador, freio, volante, marcha) do carro com autoridade	Não confiável	30 Hz
AuthorityRequest / Grant / Release	Confiável	Por evento
PartEvent (instalou, removeu, parafusou, apertou X de Y)	Confiável e ordenado	Por evento
Spawn / Despawn (compras, itens consumidos)	Confiável	Por evento
WorldState (hora, clima, eventos)	Confiável	1 Hz e por evento
Hello / Version / Ping	Confiável	Conexão e a cada 1 s

SEÇÃO 07

Duplicação do mundo do jogador

A parte inédita do projeto. Ordem sugerida: casa, variáveis, garagem, Satsuma.

7.1 Casa clonada (R4)

A casa e o quintal ficam sob a hierarquia YARD. O clone acontece no OnLoad, nos dois PCs, com o mesmo deslocamento, para que os NetIds batam:

```
var original = GameObject.Find("YARD");
var copy = Object.Instantiate(original);
copy.name = "YARD_P2";
copy.transform.position = original.transform.position + offset;
copy.transform.rotation = original.transform.rotation * Quaternion.Euler(0f, 180f, 0f);
```

Implementação em três passos, cada um entregável sozinho:

Passo	O que fazer	Resultado
1. Casca	Clonar e remover todos os PlayMakerFSM da cópia	Casa sólida e visitável, sem lógica
2. Sistemas	Religar um por um: portas, luz, chuveiro, pia, geladeira, fogão, sauna, cama, telefone, contas	Casa funcional do jogador 2
3. Persistência	Estado da cópia (luz, contas, itens) no save do jogador 2	Casa 2 sobrevive entre sessões

Para cada sistema há duas saídas: **clonar a FSM e redirecionar** (trocar referências globais e buscas por nome com AddVariable/FsmInject) ou **reescrever em C#** quando a FSM é pequena. Na prática, sistemas simples (luz, porta) funcionam clonados; sistemas ligados a variáveis globais (comer, dormir, contas) pedem o PlayerContext do item 7.3.

7.2 Mapa (R10)

- A frente da casa precisa de terreno plano e acesso de carro. O terreno do MSC é malha; a solução é colocar uma **malha nova por cima** (platô, caminho, garagem) via AssetBundle, ou ajustar colliders.
- Objetos do próprio jogo (cercas, postes, placas) são reaproveitados com Instantiate, como a casa.
- Modelos **novos** são feitos no Blender, importados no Unity 5.0.0f4 e empacotados em AssetBundle.
- **Regra legal**: nunca exportar um modelo do jogo e redistribuí-lo dentro do bundle. Tudo que vem do jogo é clonado em runtime, na cópia de quem joga.

7.3 Variáveis por jogador: PlayerContext (R2)

Cada PC só controla um corpo: o jogador local. Então as variáveis globais do jogo podem continuar significando 'o jogador local', desde que cada PC carregue os valores **do seu jogador**. Isso simplifica muito:

- No PC do jogador 2, as globais (dinheiro, fome etc.) passam a ser as do jogador 2, lidas do save do mod, não do save do jogo.
- O PC 2 envia esses valores ao PC 1 (para HUD, telefone, debug) e salva no save dele.
- O cuidado está nos sistemas do mundo que **leem ou escrevem** essas globais por conta própria: contas que chegam pelo correio, pagamentos no Teimo, prêmios de rally, multas. Esses pontos recebem

FsmInject/Harmony para descobrir **quem** é o jogador envolvido e aplicar o efeito no contexto certo.

- Contas e correspondência ficam duplicadas: caixa de correio na casa 2, ligadas ao jogador 2.

7.4 Satsuma (R3, R7)

A Satsuma é o sistema mais pesado do jogo: centenas de peças, cada uma com FSMs de instalada/solta, parafusos e desgaste, e muitas referências por nome ao carro. Por isso ela é a última grande fase.

- **Abordagem**: clonar o carro **já desmontado** no estado inicial e clonar o conjunto de peças soltas iniciais, com nomes de objetos únicos para a cópia.
- Mapear todas as FSMs de peças (lista gerada automaticamente por uma ferramenta de dump do próprio mod) e anotar quais usam busca por nome. Essas recebem a referência correta por AddVariable/patch.
- Sincronização: estado de cada peça por evento (PartEvent), física do carro por snapshot do PC com autoridade.
- Plano B se a clonagem da Satsuma se mostrar inviável: jogador 2 usa outro veículo do jogo como 'carro do projeto' numa primeira versão, enquanto a Satsuma 2 amadurece.

7.5 Garagem (R5)

A garagem faz parte do terreno da casa, então vem junto com o clone do YARD. Ferramentas, macaco, carregador de bateria e itens de oficina são duplicados como itens do jogador 2. A oficina do Fleetari (NPC) não é duplicada fisicamente: ela ganha uma **fila de serviços por jogador**.

7.6 Save (R12)

O que	Onde fica salvo
Mundo, casa 1, Satsuma 1, jogador 1	Save normal do jogo, no PC do jogador 1 (host da sessão)
Variáveis do jogador 2	Save do mod (SaveLoad), no PC do jogador 2
Casa 2, Satsuma 2, itens do jogador 2	Save do mod, no PC do jogador 2; enviado ao host ao conectar
Quem é dono de cada item comprado	Save do mod dos dois lados, por NetId

Assim cada um leva o próprio progresso. Se trocarem de host, quem hospeda carrega o mundo e o outro traz o seu 'pacote de jogador'.

SEÇÃO 08

Rede, lag e física

O que esperar de cada situação e as técnicas usadas para esconder a latência.

8.1 O que cada jogador sente

Situação	Sensação	Técnica
Mexer nas suas coisas	Instantâneo	Simulação local (você tem a autoridade)
Ver o outro andar e dirigir	Suave, ~80 a 150 ms no passado	Interpolação com buffer
Racha lado a lado	Outro carro quase no lugar real	Extrapolação (dead reckoning) com velocidade
Dirigir o carro do outro	Instantâneo; pequeno ajuste na troca	Autoridade temporária + suavização
Batida carro x carro	Consistente na maioria dos casos	Autoridade de colisão por ~0,5 s
Os dois puxando o mesmo item	Um ganha; o outro sente leve atraso	Regra de desempate (quem pegou primeiro)

8.2 Distância equivalente a 100 ms de atraso

Velocidade	60 km/h	100 km/h	150 km/h
Diferença sem extrapolação	~1,7 m	~2,8 m	~4,2 m

8.3 Por que não servidor dedicado

Servidor não reduz o lag (o caminho fica A, servidor, B) e, para simular a física, teria que rodar o próprio My Summer Car numa máquina Windows na nuvem, com licença e sem garantia de rodar sem tela. O relay gratuito da Steam já resolve conexão e protege o IP. Servidor próprio só faria sentido para mundo persistente ou mais de 2 jogadores.

8.4 Detalhes de implementação

- Snapshots com timestamp do remetente e relógio sincronizado por ping (offset estimado continuamente).
- Buffer de interpolação adaptativo (cerca de 2x o intervalo de envio mais o jitter medido).
- Corpo remoto: Rigidbody cinemático movido por MovePosition/MoveRotation no FixedUpdate; ao ganhar autoridade, vira dinâmico com a velocidade recebida.
- Correção suave: erro pequeno é corrigido em alguns frames; erro grande (teleporte) é aplicado de uma vez.
- Envio só de objetos acordados e próximos (relevância por distância e por movimento) para economizar banda.
- Quantização de posição e rotação para reduzir o tamanho dos pacotes.

SEÇÃO 09

Mundo compartilhado x individual

O que existe uma vez só e o que existe por jogador. Os itens marcados 'decidir' pedem uma escolha de vocês.

Sistema	Proposta	Observação
Hora do dia e clima	Compartilhado (host manda)	Os dois precisam ver o mesmo céu
Tráfego e NPCs	Compartilhado (host simula)	Cliente vê réplicas
Polícia e multas	Compartilhado; multa vai para quem foi pego	Usa o PlayerContext
Loja do Teimo: estoque	Decidir	Compartilhado é mais simples; individual é mais 'justo'
Loja do Teimo: compra	Individual (dinheiro de quem paga)	Item comprado nasce com dono
Encomendas pelo correio	Individual	Entrega na casa do comprador
Fleetari (oficina NPC)	Fila por jogador	Serviço cobrado do dono do carro
Inspeção	Individual, por carro	Resultado salvo no dono
Rally	Evento compartilhado, resultado individual	Seção 10
Necessidades (fome, sede, sono...)	Individual	Cada PC simula as do jogador local
Contas (luz, telefone)	Individual, por casa	Casa 2 tem contas próprias
Morte / permadeath	Decidir	Afeta só quem morreu? Ou game over geral?
Dormir e passar tempo	Decidir	O tempo só avança se os dois dormirem, ou só acelera para quem dormiu?

Atenção: dormir
O sono é o ponto mais delicado: no jogo original, dormir avança o relógio do mundo. Com dois jogadores, a opção mais simples é exigir que os dois estejam na cama (como em outros jogos co-op). Alternativa: quem dorme recupera energia sem avançar o mundo.

SEÇÃO 10

Rally e eventos (R9)

- O evento de rally acontece no mundo compartilhado: os dois podem se inscrever e correr juntos.
- Cada PC cronometra o **próprio carro** (checkpoints e linha de chegada detectados pelo PC com autoridade do carro). O resultado é enviado ao outro com o timestamp sincronizado.
- Classificação, prêmio em dinheiro e progresso de rally vão para o PlayerContext de cada um.
- Rachas livres usam a mesma regra: chegada decidida pelo dono do carro, nunca pelo que aparece na tela do outro.
- Contato roda a roda em alta velocidade continua sendo o caso menos preciso, como em todo jogo de corrida online.

SEÇÃO 11

Roadmap de implementação

Cada fase termina com algo testável. A estimativa considera uma pessoa, em tempo parcial.

Fase	Entregas	Pronto quando	Estimativa
0. Base	Ambiente (VS, MSCLoader dev mode, dnSpy), mod vazio, ferramenta de dump de FSMs e de hierarquia	Dump das FSMs da casa e da Satsuma gerado em arquivo	1 a 2 semanas
1. Casca da casa	Clone do YARD sem FSMs, posicionado; terreno plano provisório	Casa 2 visível, sólida, sem bugs de static batching	1 a 2 semanas
2. Rede mínima	Lobby Steam, Hello/Ping, PlayerState, avatar do outro jogador, interpolação	Dois PCs se veem andando com ping na tela	2 a 4 semanas
3. Física e autoridade	Registry/NetId, BodySnapshot, autoridade de itens soltos, pegar e jogar itens	Um joga uma caixa e o outro pega, sem duplicar	3 a 5 semanas
4. Veículos	Sincronizar carros prontos do jogo (van, trator), entrar no carro do outro, colisão	Racha com van, batida consistente	3 a 5 semanas
5. PlayerContext	Variáveis por jogador, save do jogador 2, compras com dono, HUD correto	Cada um com dinheiro e fome próprios após reiniciar	4 a 6 semanas
6. Casa 2 funcional	Religar sistemas da casa clonada; contas e correio da casa 2	Jogador 2 vive só na casa 2	4 a 6 semanas
7. Mundo	Hora, clima, tráfego, Teimo, Fleetari, polícia, regras de sono e morte	Um dia inteiro jogado juntos sem dessincronizar	3 a 5 semanas
8. Satsuma 2	Clone da Satsuma e peças, PartEvent, sincronia de montagem	Os dois montam carros diferentes; um mexe no carro do outro	8 a 14 semanas
9. Rally	Inscrição, cronometragem pelo dono, prêmios individuais	Rally corrido junto com resultados corretos	2 a 3 semanas
10. Polimento	Reconexão, desync recovery, configurações, release no Nexus/GitHub	Sessão de 3 h sem precisar reiniciar	contínuo

Ordem pensada para validar o maior risco cedo: a Fase 1 testa a ideia do clone; as Fases 2 a 4 provam a rede com veículos simples antes de encarar a Satsuma. Total estimado: **9 a 14 meses** em tempo parcial.

Estrutura sugerida do repositório

```
msc-duo/
  src/MscDuo/
    MscDuo.cs              entrada do mod (ModSetup)
    Net/                   Transport, Protocol, mensagens
    Sync/                  Registry, Ownership, Replication
    Game/                  hooks de FSM e patches Harmony
    Player/                PlayerContext, save do jogador
    Duplication/           clone da casa, garagem, Satsuma
    World/                 hora, clima, lojas, rally
    Tools/                 dump de FSM/hierarquia (só em dev)
  unity/                   projeto Unity 5.0.0f4 para AssetBundles
  docs/                    este plano, mapas de FSM
  LICENSE                  GPL v3
```

SEÇÃO 12

Riscos técnicos

Risco	Prob.	Impacto	Mitigação / plano B
Static batching quebra o clone da casa	Média	Alto	Validar na Fase 1; reconstruir malhas da cópia
FSMs da Satsuma com referências por nome demais	Alta	Muito alto	Dump automático + patch em massa; plano B com outro veículo
Física divergente em batidas	Alta	Médio	Autoridade de colisão; aceitar imperfeição em contato rápido
Biblioteca de rede incompatível com .NET 3.5 / steam_api do jogo	Média	Alto	Testar na Fase 2; alternativa LiteNetLib (versão compatível) + relay
Atualização do jogo quebra hooks	Média	Médio	Hooks por nome de estado com checagem; testes de fumaça
Desempenho (muitos objetos sincronizados)	Média	Médio	Relevância por distância, objetos dormindo não enviam
Conflito com outros mods	Alta	Baixo	Declarar incompatibilidades; modo 'só MSC Duo'
Escopo grande demais e desânimo	Alta	Alto	Fases curtas com resultado visível; Satsuma por último

SEÇÃO 13

Análise legal

Vocês podem ser barrados de fazer isso? Resumo prático, com a ressalva de que isto não é aconselhamento jurídico.

Conclusão
Fazer e distribuir gratuitamente um mod que roda sobre uma cópia legítima do jogo, sem redistribuir arquivos do jogo, é prática comum e tolerada há anos na comunidade de My Summer Car. O risco de ser barrado existe, mas é **baixo** se as regras abaixo forem seguidas.

13.1 O que foi verificado

- **Termos do jogo**: a página da Steam (Amistech Games, lançamento completo em 8/1/2025) não traz EULA própria; valem os termos gerais da Steam (Steam Subscriber Agreement). Não foi encontrada política oficial da Amistech proibindo mods.
- **Histórico**: MSCMP, BeerMP, WreckMP e MSC Online existiram publicamente. A única disputa conhecida (BeerMP) foi de licença **entre modders**, não com a desenvolvedora.
- **MSCLoader**: GPL v3. Mods que dependem dele podem ser publicados como GPL v3 sem conflito.
- **MSCMP**: GPL v3. Pode servir de base, desde que o derivado continue GPL v3 e dê créditos.
- **Lei brasileira**: a Lei 9.609/98 (software) e a Lei 9.610/98 (direitos autorais, que cobre arte e modelos) protegem o código e os assets do jogo. O mod é obra própria que interage com o jogo; o risco aparece quando há **cópia** ou **redistribuição** do que é deles.

13.2 Linhas vermelhas

Não fazer	Por quê	Como o plano evita
Redistribuir DLLs, cenas ou assets do jogo (inclusive modelos exportados)	Cópia de obra protegida	Clone da casa e da Satsuma feito em runtime, na cópia de quem joga
Facilitar pirataria	Principal motivo de reação de desenvolvedores	Exigir sessão Steam e cópia legítima nos dois PCs
Cobrar pelo mod ou por acesso antecipado	Transforma uso tolerado em exploração comercial	Distribuição gratuita; doação opcional, no máximo
Copiar código de mods sem licença (WreckMP, MSCO)	Violação de direito autoral do modder	Código autoral; só MSCMP/MSCLoader (GPL) como referência
Usar marca/logo do jogo como se fosse oficial	Confusão com produto oficial	Nome próprio e aviso 'mod não oficial'

13.3 Boas práticas

- Publicar o código no GitHub como GPL v3, com créditos ao MSCLoader e, se usado, ao MSCMP.
- Avisar no README e na página do mod que é um projeto de fãs, sem afiliação com a Amistech Games.
- Se o projeto crescer, mandar uma mensagem educada à Amistech apresentando o mod. Uma resposta positiva elimina a dúvida.

- Responder rápido se a desenvolvedora pedir alguma mudança ou remoção.
- Declarar uso de IA conforme pede o MSCLoader, caso algum código seja gerado por IA.

SEÇÃO 14

Ferramentas, setup e próximos passos

Ferramenta	Para quê
My Summer Car (Steam) + MSCLoader com dev mode	Rodar e depurar o mod
Visual Studio + templates do MSCLoader	Projeto C# para .NET 3.5
dnSpy / ILSpy	Ler o Assembly-CSharp e as DLLs do PlayMaker
Mods de inspeção/dev (dev mode do MSCLoader)	Ver hierarquia e FSMs em runtime
Unity 5.0.0f4	Gerar AssetBundles compatíveis
Blender	Modelos novos do mapa
Steamworks.NET (versão compatível com .NET 3.5) ou LiteNetLib	Rede P2P; validar na Fase 2
Duas contas Steam com o jogo	Testar a sessão de verdade (dois PCs, ou um PC e uma VM)
GitHub	Código aberto (GPL v3), issues, releases

Próximos passos imediatos

- 1. Confirmar onde o jogo está instalado e instalar o MSCLoader.
- 2. Criar o repositório `msc-duo` em Documents\Codes, com GPL v3.
- 3. Fase 0: mod vazio + ferramenta de dump de hierarquia e FSMs (YARD e Satsuma).
- 4. Fase 1: clonar a casca da casa e testar static batching.
- 5. Decidir os itens marcados como 'decidir' na seção 9 (estoque do Teimo, morte, sono).

Fontes consultadas

- MSCLoader: github.com/piotrulos/MSCModLoader (README, wiki e código: Mod.cs, Mod.Internal.cs, MainEntry.cs, SaveLoad.cs, PlayMakerExtensions.cs)
- MSCMP OLD: github.com/RootKiller/MSCMP-OLD (README e licença GPL v3); mscmp.itch.io/mscmp
- WreckMP: nexusmods.com/mysummercar/mods/5271 e discussão na Steam (steamcommunity.com/app/516750/discussions/2/4357872562127095970)
- Página da Steam: store.steampowered.com/app/516750/My_Summer_Car
- Lei 9.609/98 (Lei do Software) e Lei 9.610/98 (Direitos Autorais), planalto.gov.br